

RRAM Device Modelling Using COMSOL Multiphysics and MATLAB LiveLink

A Report Submitted in Partial Fulfilment of the Requirements for the

SN Bose Internship Program, 2026

Submitted by

Sweta Maurya
Scholar ID: 2315080

Under the guidance of

Dr. Shivendra Kumar Pandey

Assistant Professor

Department of Electronics and Instrumentation Engineering
National Institute of Technology Silchar



Department of Electronics and Instrumentation Engineering
NATIONAL INSTITUTE OF TECHNOLOGY SILCHAR
Assam

June-July, 2026

DECLARATION

I hereby declare that the work presented in this report, titled **"RRAM Device Modelling Using COMSOL Multiphysics and MATLAB LiveLink"**, submitted in partial fulfilment of the requirements for the SN Bose Internship Program, 2026, is a record of the work carried out by me during the internship period under the guidance of **Dr. Shivendra Kumar Pandey**, Department of Electronics and Instrumentation Engineering.

The COMSOL model development, MATLAB–COMSOL LiveLink integration, debugging of the simulation pipeline, and documentation described in this report are presented to the best of my knowledge and represent my own understanding and effort. Where external research material, theoretical frameworks, or code structures were used as reference, the corresponding sources have been duly acknowledged and cited. In particular, the thermodynamic RRAM modelling framework developed by Niraula and Karpov was used as the principal technical reference for the device modelling approach followed in this work.

I further declare that the final I-V characterization results presented in this report are preliminary and remain a work in progress at the time of submission, as stated in the Abstract, and have been represented honestly as such.

Sweta Maurya
Scholar ID: 2315080
B.Tech., 7th semester

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, **Dr. Shivendra Kumar Pandey**, Department of Electronics and Instrumentation Engineering, for his invaluable guidance, encouragement, and support throughout the course of this internship. His direction was instrumental in helping me understand the scope of the project and navigate the technical challenges encountered along the way.

I am also thankful to **Mr. Arijit** (PhD Scholar) for helping me understand the underlying physics of RRAM device operation, and to **Mr. Chandrashekhar** (JRF) for his patient help in learning the COMSOL Multiphysics software environment. Their guidance was of great help in building the foundation needed to carry out this work.

I would also like to thank the Department of Electronics and Instrumentation Engineering for providing the computational resources and laboratory environment necessary for this project, and everyone in the lab who supported me through discussions and troubleshooting during the internship period.

Sweta Maurya
Scholar ID: 2315080
B.Tech., 7th semester

ABSTRACT

Resistive Random-Access Memory (RRAM) is a non-volatile memory technology in which the resistance of a metal-insulator-metal structure is switched through the formation and rupture of a nanoscale conductive filament. This internship focused on numerical modelling of filamentary RRAM switching using COMSOL Multiphysics 6.0 coupled with MATLAB, based on the thermodynamic modelling framework developed by Niraula and Karpov. A two-dimensional axisymmetric RRAM structure comprising TiN electrodes, HfO_2 , Hf, HfO_{2-x} conductive filament, switching gap, and surrounding SiO_2 regions was developed using coupled Heat Transfer in Solids, Electric Currents, and Electrical Circuit physics.

Four COMSOL modules — RRAM_OFF, RRAM_ON, RRAM_SET, and RRAM_RESET were implemented to represent the principal device states and switching transitions. COMSOL–MATLAB LiveLink was used to automate parameter sweeps, repeated simulations, and extraction of electrical and thermal quantities. A major part of the work involved debugging the inherited COMSOL–MATLAB codebase, including correcting mismatched geometry domain numbers, parameter naming conflicts, MATLAB syntax errors, and memory-management issues that interrupted long parameter sweeps.

At the time of writing, the simulation framework successfully runs through model construction and parameter sweeping without errors. However, the final I–V characteristics have not yet been validated against the switching behaviour reported in the literature, and this validation remains ongoing. The internship therefore resulted in a functional and debugged COMSOL–MATLAB framework for filamentary RRAM modelling, providing a foundation for obtaining and validating the final device characteristics.

LIST OF FIGURES

Figure	Description	Page
1.1	MIM RRAM structure — cross-section and 3D view	2
2.1	Resistive switching mechanism — forming, SET, RESET	4
3.1	Labeled cross-section of RRAM device geometry	8
3.2	Surface-to-Ambient Radiation boundary conditions	10
3.3	Free triangular mesh for RRAM_SET	11
6.1	3D temperature distribution, RRAM_OFF	22
6.2	Combined device I-V scatter plot	25
6.3	Stage-wise I-V characteristic (log scale)	26
8.1	Electric potential distribution, RRAM_SET	32

LIST OF TABLES

Table	Description	Page
3.1	Material properties assigned to device domains	9
4.1	MATLAB post-processing functions	15
5.1	Summary of debugging issues resolved	21
6.1	Output tables generated per simulation run	23

CONTENTS

	Page
Chapter 1 Introduction	1
Chapter 2 Literature Review and Theoretical Background	4
Chapter 3 COMSOL Device Model: Geometry, Materials, and Physics	7
Chapter 4 MATLAB–COMSOL LiveLink Integration and Simulation Workflow	13
Chapter 5 Debugging the Inherited Simulation Pipeline	17
Chapter 6 Results	22
Chapter 7 Data Processing and I-V Characterization	28
Chapter 8 Current Status and Limitations	31
Chapter 9 Learning Outcomes and Future Scope	34
Chapter 10 Conclusion	36
References	37

Chapter 1

Introduction

1.1 Background and Motivation

The rapid growth of data-intensive computing, edge devices, and neuromorphic computing systems has driven the search for memory technologies that can offer higher density, faster switching, lower power consumption, and non-volatility, going beyond the physical scaling limits of conventional Flash memory. Among the various emerging non-volatile memory (NVM) technologies — including Phase Change Memory (PCM), Magnetoresistive RAM (MRAM), and Ferroelectric RAM (FeRAM) — Resistive Random-Access Memory (RRAM) has attracted significant research interest due to its simple metal-insulator-metal (MIM) structure, CMOS-compatible fabrication, fast switching speed, low operating voltage, and scalability to nanometer dimensions.

RRAM operates on the principle of resistive switching, wherein the resistance state of a dielectric layer sandwiched between two metal electrodes can be reversibly changed between a High Resistance State (HRS, or OFF state) and a Low Resistance State (LRS, or ON state) by applying appropriate electrical stimuli. This switching behaviour is commonly attributed to the formation and rupture of a nanoscale conductive filament (CF) within the dielectric layer, driven by the migration of oxygen vacancies or metal ions under an applied electric field, coupled with local Joule heating effects.

Despite considerable experimental progress, a complete physical understanding of the microscopic mechanisms governing filament formation, growth, and rupture — and particularly their dependence on factors such as voltage ramp-rate and cycle-to-cycle variability — remains an active area of research. Numerical and physics-based modelling of RRAM devices therefore plays an important role in bridging experimental observations with the underlying thermodynamic and electrothermal mechanisms, and in guiding device design and optimization.

1.2 Resistive Random-Access Memory (RRAM)

A typical RRAM cell consists of a thin dielectric switching layer (commonly a transition metal oxide such as HfO_2 , TiO_2 , or Ta_2O_5) sandwiched between a top and bottom metal electrode. Application of a sufficiently large positive voltage across the device induces the formation of a conductive filament — composed of oxygen vacancies or a sub-stoichiometric oxide phase (e.g., HfO_{2-x}) — that bridges the two electrodes, switching the device from the HRS to the LRS. This process is referred to as the **SET**

operation. Conversely, application of a voltage of opposite polarity (or, in some device types, the same polarity at higher magnitude) ruptures the filament, reintroducing a resistive gap and switching the device back to the HRS. This process is referred to as the **RESET** operation.

The device's electrical behaviour is closely coupled to its thermal behaviour: the Joule heating generated by current flow through the filament directly influences the local temperature, which in turn governs the rate of ion/vacancy migration and filament growth or dissolution. This tight electro-thermal coupling, combined with the stochastic nature of filament formation at the nanoscale, gives rise to characteristic RRAM behaviours such as voltage ramp-rate-dependent switching thresholds and cycle-to-cycle variability in the device I-V characteristics — both of which are well documented in the RRAM literature and form key benchmarks for evaluating the accuracy of a numerical device model.

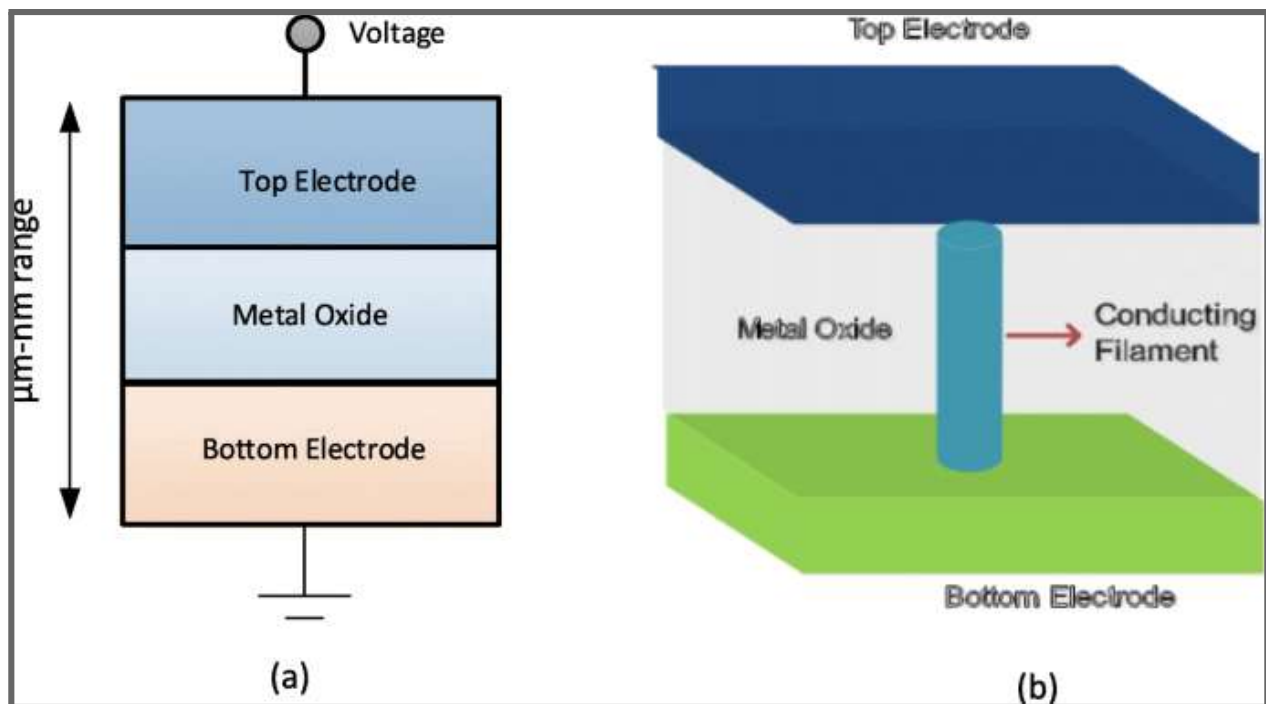


Figure 1.1: (a) Schematic cross-section of the metal-insulator-metal (MIM) RRAM structure, consisting of top and bottom electrodes separated by a metal-oxide switching layer in the μm – nm thickness range; (b) three-dimensional representation of the same structure showing the localized conducting filament formed within the metal oxide layer.[16]

1.3 Objectives of the Internship

The primary objective of this internship was to develop and evaluate a numerical model of filamentary RRAM switching, based on the thermodynamic modelling framework proposed by Niraula and Karpov, using COMSOL Multiphysics coupled with MATLAB. Specifically, the internship aimed to:

1. Understand the physical mechanisms underlying RRAM resistive switching, including filament nucleation, growth, and rupture, and the associated electro-thermal coupling.
2. Study and reproduce a four-module COMSOL device model (RRAM_OFF, RRAM_ON, RRAM_SET, and RRAM_RESET) representing the principal operational states of the device.
3. Establish a MATLAB–COMSOL LiveLink workflow to automate parameter sweeps, repeated solving of the coupled electro-thermal-circuit model, and extraction of device-level electrical and thermal quantities.
4. Implement the free-energy minimization approach used to determine the thermodynamically stable filament radius (during SET) and gap length (during RESET) at each simulation step.
5. Debug and validate the inherited simulation pipeline, and generate device I-V characteristics for comparison against literature-reported RRAM switching behaviour, including voltage ramp-rate dependence and cycle-to-cycle variation.

1.4 Organization of the Report

The remainder of this report is organized as follows. Chapter 2 reviews the relevant literature and theoretical background on RRAM switching physics and the Niraula–Karpov thermodynamic modelling framework. Chapter 3 describes the COMSOL device geometry, materials, and physics setup. Chapter 4 details the MATLAB–COMSOL LiveLink integration and simulation workflow. Chapter 5 documents the debugging process undertaken during the internship. Chapter 6 presents the results obtained to date. Chapter 7 discusses the data post-processing and I-V characterization methodology. Chapter 8 outlines the current status and limitations. Chapter 9 summarizes the learning outcomes and proposes future work, and Chapter 10 concludes the report.

Chapter 2

Literature Review and Theoretical Background

2.1 RRAM Switching Mechanisms

The resistive switching behaviour of oxide-based RRAM devices has been widely attributed to the electrochemical formation and rupture of a conductive filament (CF) within the dielectric switching layer.

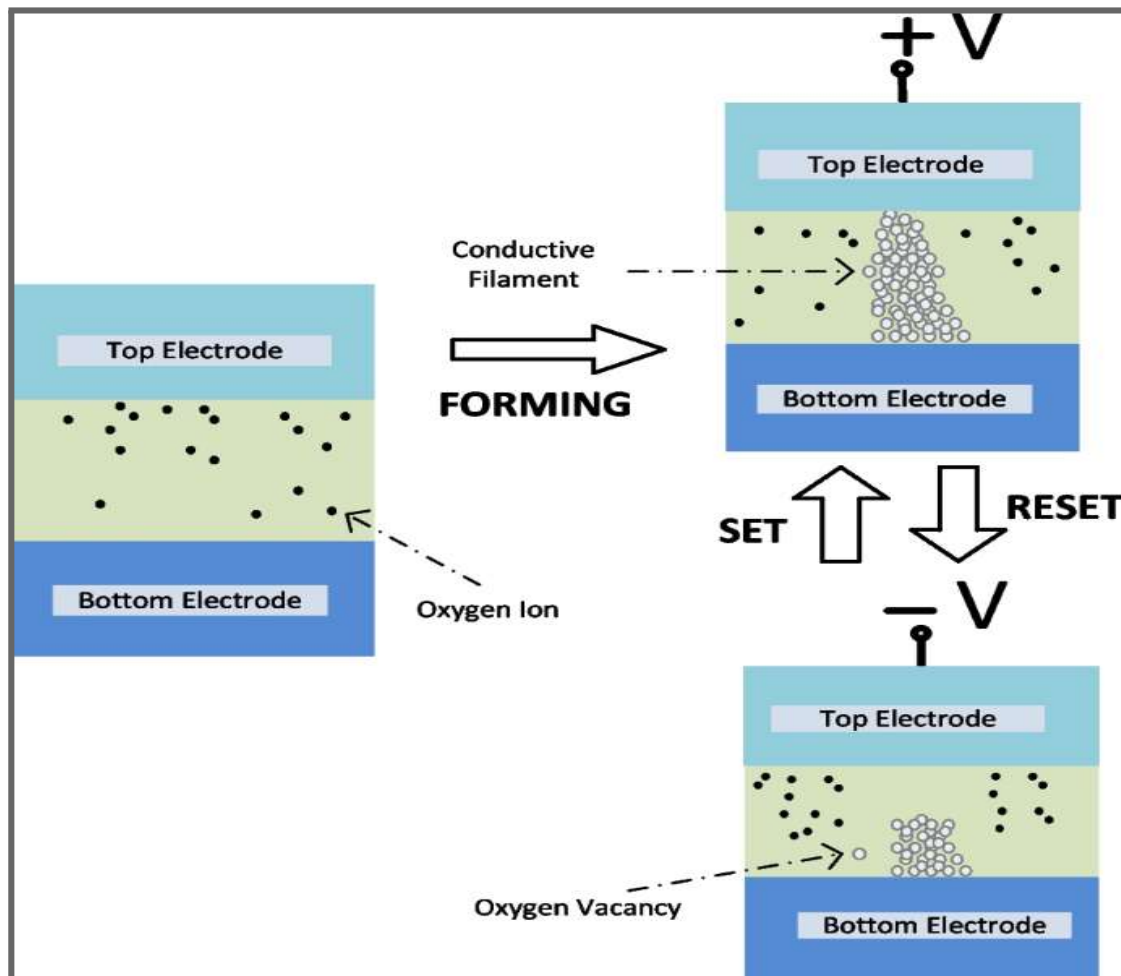


Figure 2.1: Schematic illustration of the resistive switching mechanism in an oxide-based RRAM cell, showing the electroforming process (formation of a conductive filament from oxygen ion migration), followed by SET (filament reformation, oxygen vacancy migration) and RESET (filament rupture) under opposite voltage polarities.[17]

Two switching polarities are commonly reported in the literature: **unipolar switching**, where SET and RESET occur under the same voltage polarity but different magnitudes, and **bipolar switching**, where SET and RESET require opposite voltage polarities. The device structure and framework considered in this internship follows a bipolar switching scheme, consistent with the TiN/HfO₂/HfO_{2-x}/Hf material stack used in the COMSOL model.

In oxide systems such as HfO₂, this filament is generally understood to consist of a chain of oxygen vacancies (or, equivalently, a locally sub-stoichiometric HfO_{2-x} phase) that forms a low-resistance conduction path between the top and bottom electrodes. The migration of oxygen ions/vacancies under the applied electric field, combined with the local temperature rise from Joule heating, governs the rate at which this filament nucleates, grows, and eventually ruptures.

2.2 The Niraula–Karpov Thermodynamic Modelling Framework

The primary theoretical and implementation framework used in this internship is the numerical RRAM model developed by Niraula and Karpov, which treats filament formation and rupture as a thermodynamically driven process governed by the minimization of a total free energy functional. Rather than tracking individual ion/vacancy hops explicitly, the model evaluates, at each applied voltage/current step, the total free energy of the system as a function of a single geometric order parameter — the filament radius (during the SET process) or the gap length (during the RESET process) — and identifies the thermodynamically stable configuration as the one that minimizes this free energy.

The total free energy considered in the model is composed of the following contributing terms:

- **Volume (chemical) energy** — the energy associated with the phase change of the oxide material within the filament volume, expressed in terms of a chemical potential difference, $\Delta\mu$, between the conductive and insulating phases.
- **Surface energy** — the energy cost associated with the interface between the filament and the surrounding dielectric, proportional to the filament's lateral surface area and a surface tension coefficient.
- **Electrostatic energy** — the energy stored in the electric field within the dielectric/gap region, computed from the simulated electric field distribution.
- **Thermal energy** — the sensible heat energy stored in the electrode, filament, dielectric, and Hf-layer regions due to Joule heating, evaluated relative to a reference (room) temperature.

For a given applied bias, these energy terms are evaluated over a range of trial filament radii (or gap lengths), and the configuration corresponding to the minimum total free energy is located — first by a coarse sweep to bracket the minimum, followed by a golden-section/Brent's method refinement for faster convergence. This stable radius (or gap length) is then carried forward as the initial condition for the next voltage/current step, allowing the model to trace out the evolution of the filament across a full switching cycle.

2.3 Filament Nucleation and Threshold Voltage

Prior to filament growth, the model accounts for the field-induced nucleation of a needle-shaped filament embryo within the dielectric. Nucleation is treated as a thermally activated process governed by a nucleation barrier energy, with the associated threshold voltage derived as a function of the applied ramp-rate, the atomic vibration timescale, the critical nucleation radius, and the local gap-region temperature. This ramp-rate dependence is a key feature the model aims to reproduce, and is one of two principal benchmarks — alongside cycle-to-cycle variability — used to validate the framework against reported measurements.

2.4 Electro-Thermal Coupling

The free-energy terms above depend on the spatially resolved electric field and temperature distributions within the device, obtained by solving the coupled electro-thermal boundary value problem. This is the role of the COMSOL finite-element model: for each trial filament geometry, COMSOL solves the coupled Electric Currents and Heat Transfer in Solids physics (via an Electromagnetic Heating coupling) together with an external Electrical Circuit representing the source and load resistor, returning the resulting fields to MATLAB for the free-energy evaluation. This coupling is what necessitated the repeated, automated LiveLink workflow described in Chapter 4

2.5 Summary

The theoretical framework reviewed in this chapter — thermodynamic free-energy minimization coupled to a finite-element electro-thermal device model — forms the basis of the four-module COMSOL model (RRAM_OFF, RRAM_ON, RRAM_SET, RRAM_RESET) and the MATLAB control scripts ([Complete_I_V_avg](#), [Complete_I_V_ramprate](#), [Complete_I_V_C2C](#)) used and debugged during this internship, which are described in detail in the following chapters.

Chapter 3

COMSOL Device Model: Geometry, materials, and Physics

3.1 Overview

The RRAM device was modelled in COMSOL Multiphysics 6.0 as a two-dimensional axisymmetric structure, exploiting the cylindrical symmetry of the physical device around its central vertical axis. This approach significantly reduces computational cost compared to a full 3D model while still capturing the radial dependence of the electric field, current density, and temperature distribution — which is essential, since the free-energy minimization described in Chapter 2 explicitly depends on the filament radius.

The complete device stack, from bottom to top, consists of: a bottom SiO_2 substrate, a bottom TiN electrode, a Hf metal layer, a switching layer containing the HfO_{2-x} conductive filament and HfO_2 dielectric side-by-side, a top TiN electrode, and a top SiO_2 layer, with an additional SiO_2 region surrounding the electrode stack laterally. During OFF and RESET states, an additional thin "Gap" domain is introduced within the filament region to represent the ruptured, insulating portion of the filament.

3.2 Geometry Construction

The geometry was built using the axisymmetric 2D workplane (`geom1`), with all dimensions specified in nanometres. Each device region was constructed as a rectangle (`Rectangle` primitive) positioned along the radial (r) and axial (z) directions, using a set of global parameters (e.g., filament radius `r_f`, device radius `r_d`, gap height `h_g`, dielectric height `h_di`) so that the geometry could be parametrically rebuilt for each trial filament radius or gap length during the free-energy minimization sweep. The rectangles were finalized using a **Form Union** operation, which merges the individual rectangles into a single conformal mesh domain set while preserving each region as a separate material domain.

For the OFF and RESET modules specifically, the filament rectangle was split into two parts — a shrunk conductive filament segment and a newly introduced "Gap" rectangle — to represent the physical break in the filament that defines the high-resistance state.

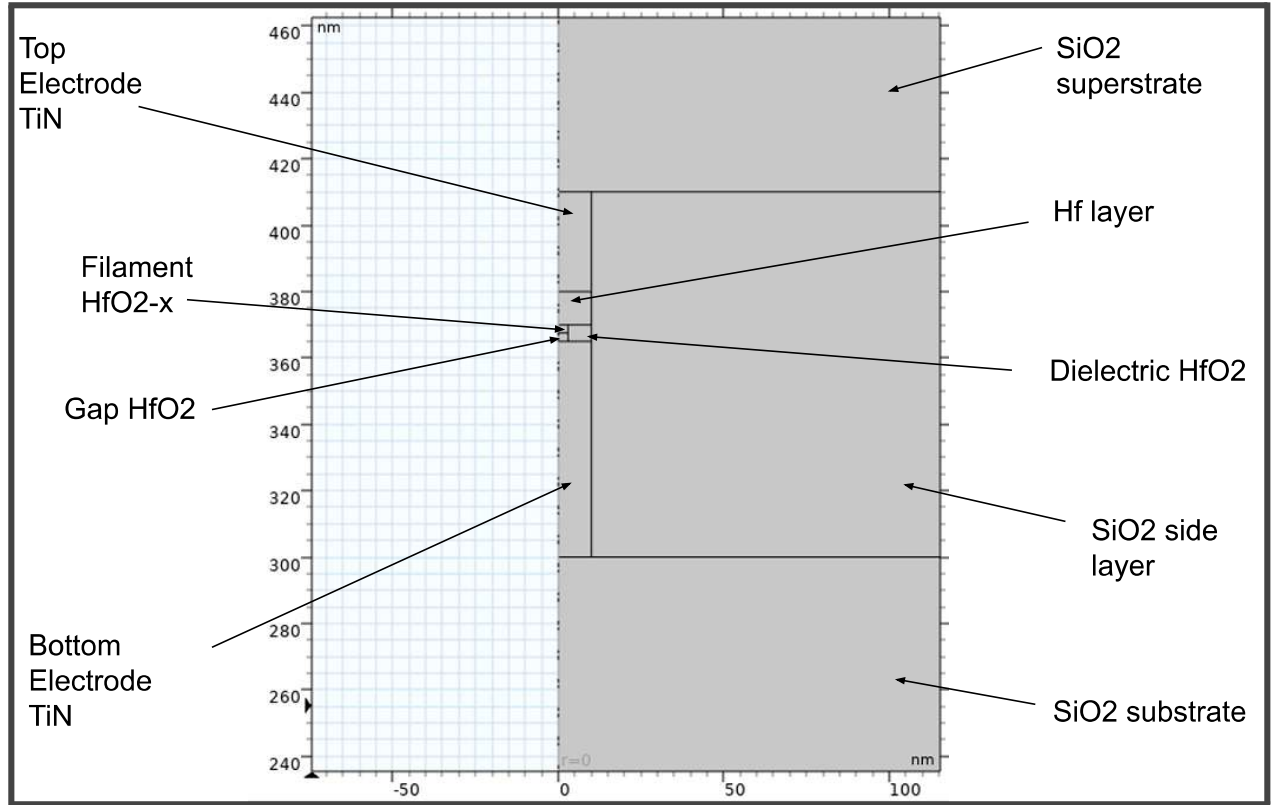


Figure 3.1: Labeled 2D axisymmetric cross-section of the RRAM device geometry showing all material domains .

3.3 Material Assignment

Table 3.1 summarizes the materials assigned to each domain of the finalized device geometry, along with the key thermal and electrical properties used.

Table 3.1: Material properties assigned to device domains

Domain / Region	Material	Density (kg/m ³)	Heat Capacity (J/kg·K)	Thermal Conductivity (W/m·K)	Electrical Conductivity (S/m)
Top / Bottom Electrode	TiN	5.22×10^3	545.33	11.9	1×10^6
Hf Layer	Hf	13.3×10^3	144	23	5×10^6
HfO _{2-x} Filament (conducting)	HfO _{2-x}	12×10^3	140	0.65	2×10^4
Gap (insulating phase)	HfO ₂ (insulating)	12×10^3	140	5.08	1.5×10^4
HfO ₂ Dielectric	HfO ₂	10×10^3	120	0.5	10
SiO ₂ (side / top / bottom)	SiO ₂	2.2×10^3	703	1.38	≈ 0 (insulating)

The relative permittivity of the HfO₂ dielectric was set to 25, consistent with reported values for hafnium oxide thin films, and used in the electrostatic energy term of the free-energy calculation described in Section 2.2.

3.4 Physics Interfaces

Three physics interfaces were defined within each COMSOL module, coupled through a multiphysics node:

(a) Heat Transfer in Solids (ht) — solves the steady-state/transient heat conduction equation across all device domains, with a fixed-temperature boundary condition (298 K) applied at the outer electrode contacts and a Surface-to-Ambient Radiation boundary condition applied to the exposed outer surfaces of the device (emissivity = 0.9, ambient temperature = 298 K).

(b) Electric Currents (ec) — solves for the electric potential and current density distribution across the conductive domains, with a Terminal boundary condition (circuit-coupled) applied at the top electrode contact and a Ground boundary condition applied at the bottom electrode contact.

(c) Electrical Circuit (cir) — represents the external drive circuit as a lumped-element network, consisting of a voltage or current source (depending on the module — DC/pulse voltage source for SET/RESET/ON, current source for the OFF-state characterization), a series load resistor (**R1**, 3.1 k Ω), and a Model-Device I-V element (**IvsU1**) that couples the circuit to the terminal voltage/current of the finite-element **ec** physics.

(d) Electromagnetic Heating (emh1) — a multiphysics coupling node that feeds the Joule heating computed from the **ec** physics as a distributed heat source into the **ht** physics, closing the electro-thermal feedback loop described in Section 2.4.

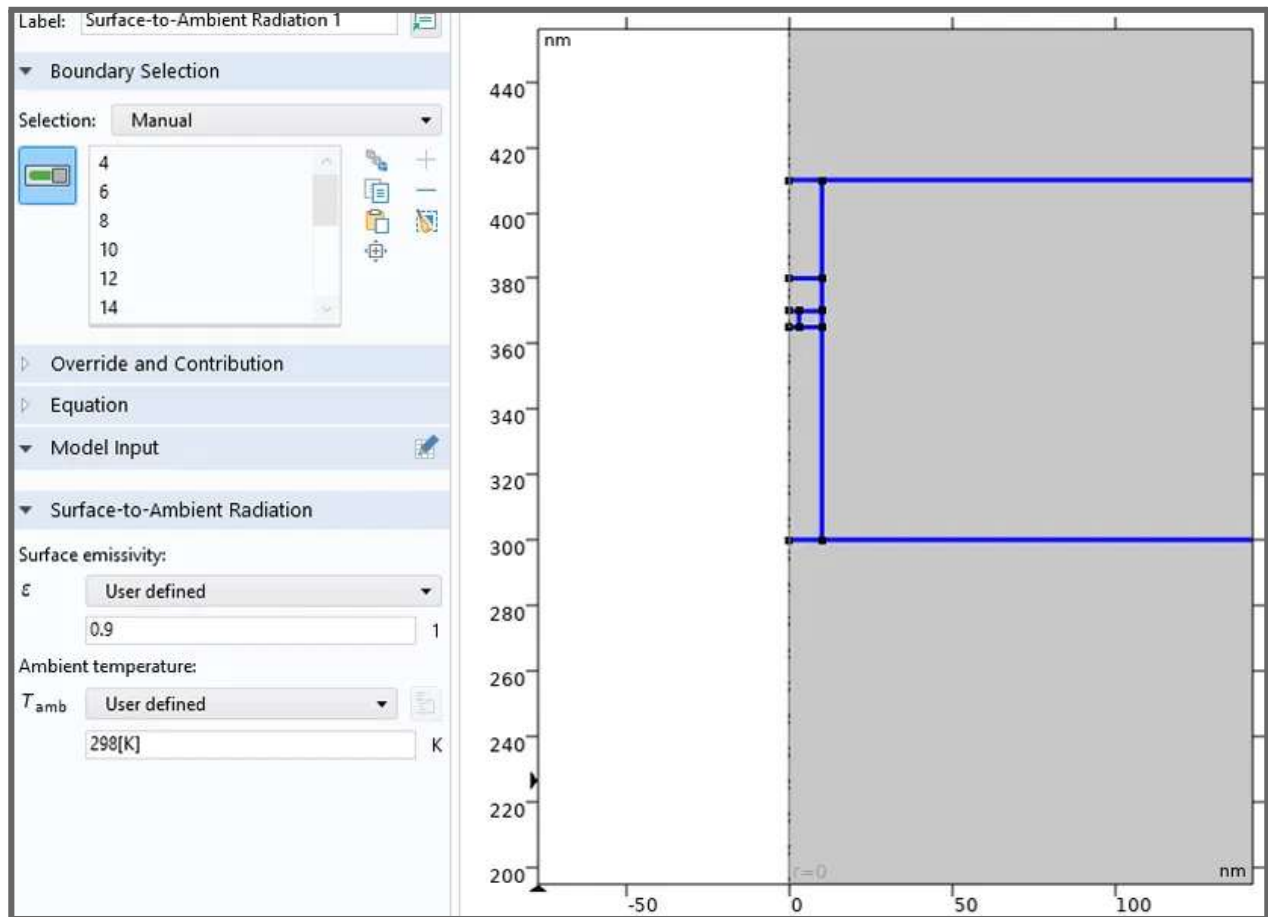


Figure 3.2: Surface-to-Ambient Radiation boundary condition settings applied to the exposed outer surfaces of the device (emissivity = 0.9, ambient temperature = 298 K).

3.5 Mesh

A free triangular mesh was used throughout the device geometry, with the default "Physics-controlled mesh" setting supplemented by a custom element-size override applied to the electrode domains to better resolve the sharp field/temperature gradients near the filament-electrode interface. Mesh convergence was checked qualitatively by inspecting the temperature and electric field distributions for mesh-dependent artifacts before finalizing the element size settings used in the production runs.

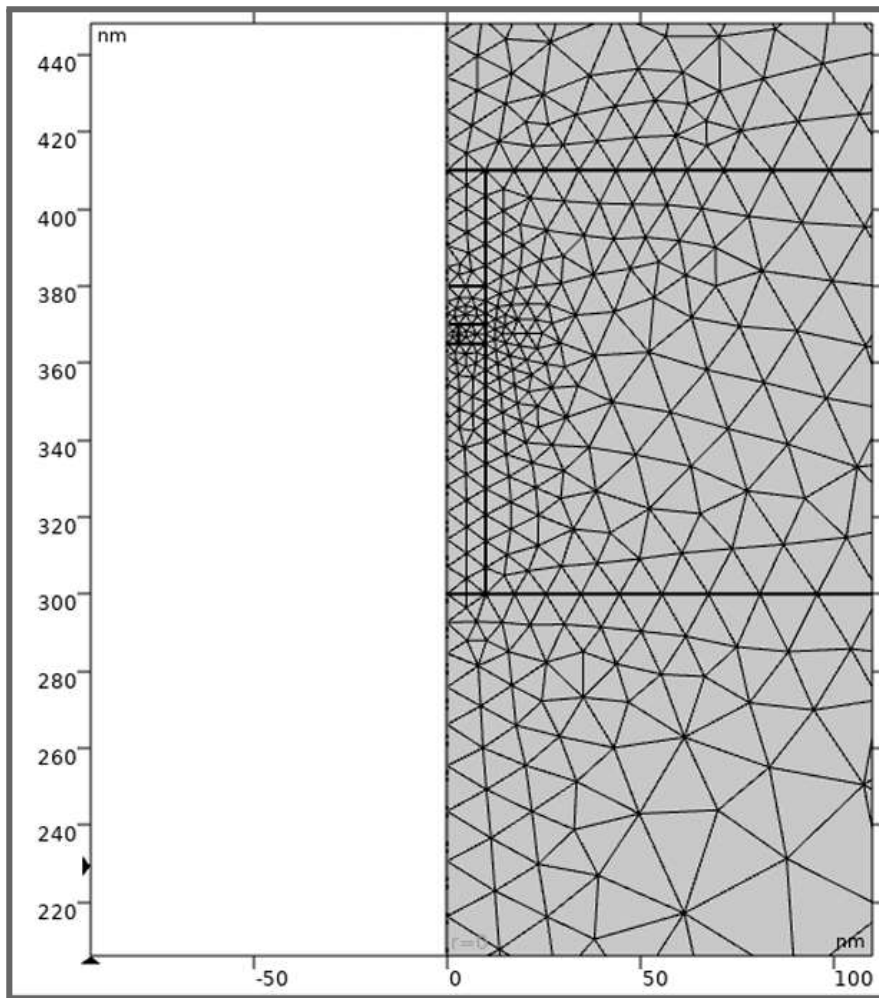


Figure 3.3: Free triangular mesh generated for the RRAM_SET module, showing finer element density near the filament–electrode interface.

3.6 Study Configuration

Each module was solved using either a **Stationary** study (for the SET module, where each free-energy evaluation corresponds to a single DC operating point) or a **Time Dependent (Transient)** study (for the ON, OFF, and RESET modules, where the applied voltage/current pulse is ramped over a finite rise time `t_rise`, computed from the pulse amplitude and the specified ramp-rate parameter). The transient studies used an output time list of the form `range(0, t_rise/30, 3/2*t_rise)`, sampling 46 time points across one and a half pulse rise-times to capture the full transient switching behaviour.

Chapter 4

MATLAB–COMSOL Livelink Integration and Simulation Workflow

4.1 Overview

While COMSOL Multiphysics solves the coupled electro-thermal-circuit boundary value problem for a single device configuration, the thermodynamic free-energy minimization described in Chapter 2 requires this problem to be solved repeatedly — once for every trial filament radius or gap length considered during the minimization sweep, and once again for every voltage/current step in a full I-V characteristic. Manually rebuilding and re-solving the model in the COMSOL Desktop GUI for each of these evaluations would be impractical. **LiveLink™ for MATLAB** was therefore used to connect COMSOL Multiphysics to MATLAB, allowing the entire simulation to be driven programmatically.

4.2 COMSOL Server–MATLAB Connection

LiveLink for MATLAB operates in a client–server architecture. A COMSOL Multiphysics server process is launched in the background, and MATLAB connects to this server as a client, sending model-building and solving commands through the `com.comsol.model` Java API exposed to MATLAB. In this internship, the server was launched from within MATLAB itself using a system shell command:

```
cd('C:\Users\NEEM Lab\Desktop\shweta\RRAM');  
!"C:\Program Files\COMSOL\COMSOL60\Multiphysics\bin\win64\comsol.exe"  
mphserver simulink &  
pause(5); % allow the server connection to establish
```

Once the server link was established, all subsequent COMSOL model objects created within the MATLAB session (via `ModelUtil.create`) were transparently instantiated on this background server and manipulated using COMSOL's MATLAB API.

4.3 Model-Building Functions

Each of the four device states — OFF, ON, SET, and RESET — was implemented as a separate COMSOL-generated MATLAB function (`RRAM_OFF.m`, `RRAM_ON.m`, `RRAM_SET.m`, `RRAM_RESET.m`). These functions, generated directly from COMSOL Desktop via **File** → **Save As** → **Model File for MATLAB**, programmatically reconstruct

the complete model — geometry, materials, physics, mesh, and solver configuration — every time they are called, returning a live COMSOL model object:

```
model_OFF = RRAM_OFF;
```

Because each call to one of these functions rebuilds the model from scratch on the server, careful attention was required to ensure that the domain numbers referenced in mesh selections and post-processing calls (e.g. `'selection', [2,5]` for the electrode domains) remained consistent with the actual geometry produced — an issue discussed further in Chapter 5.

4.4 Parameter Sweeping

Once a model object was obtained, MATLAB was used to override COMSOL's global parameters for each simulation step, using the `param.set` method:

```
model_SET.param.set('I_S', I_S); % source current for this step
model_SET.param.set('r_f', r_f); % trial filament radius
model_SET.param.set('A', A); % rate-dependent pre-factor
model_SET.geom('geom1').run; % rebuild geometry for new r_f
model_SET.study('std1').run; % solve
```

This pattern — set parameter(s), rebuild geometry if needed, run the study — forms the core of every free-energy evaluation performed during the SET and RESET minimization sweeps (functions `FE_SET` and `FE_RESET` in `Complete_I_V_avg.m`), as well as the outer voltage-ramp and cycle-to-cycle sweeps in `Complete_I_V_ramprate.m` and `Complete_I_V_C2C.m`.

4.5 Extraction of Results

After each solve, MATLAB extracted the electrical, thermal, and field quantities required for the free-energy calculation using COMSOL's `mph`-prefixed post-processing functions:

Table 4.1: MATLAB post-processing functions

Function	Purpose
<code>mphglobal</code>	Extracts circuit-level global quantities (e.g. <code>cir.IvsU1_v</code> , <code>cir.IvsU1_i</code> , <code>cir.V1_v</code> , <code>cir.R1_v</code>)
<code>mphmean</code>	Computes the domain-averaged value of a field quantity (e.g. average filament/gap/dielectric temperature)
<code>mphmax</code>	Computes the maximum value of a field quantity over a domain
<code>mphint2</code>	Computes a domain (area) integral, used for thermal and electrostatic energy terms

For example, the average and maximum filament temperature were extracted as:

```
avg_filament_Temperature_K = mphmean(model_SET, 'T', 2, 'selection', 3);  
max_filament_Temperature_K = mphmax(model_SET, 'T', 2, 'selection', 3);
```

and the electrostatic energy stored in the dielectric domain was computed as a domain integral of the local field energy density:

```
Electro_Static_Energy_eV = 6.2415e18 * mphint2(model_SET, ...  
        '((ec.Er)^2+(ec.Ez)^2+(ec.Ephi)^2)*8.85e-12*25*2*pi*r*(1/2)', 2, 'selection', 7);
```

4.6 Free-Energy Minimization Loop

For each applied source current (SET) or source voltage (RESET) step, the corresponding free-energy function (`FE_SET` or `FE_RESET`) was evaluated over a coarse range of trial filament radii / gap lengths to bracket the minimum, followed by a **Brent's method** refinement (golden-section search combined with parabolic interpolation) to converge on the thermodynamically stable value to within a specified tolerance. This nested loop structure — outer sweep over applied bias, inner minimization over filament geometry — is the central computational pattern of the entire simulation pipeline, and is what necessitates a very large number of repeated COMSOL

solves (and correspondingly repeated model builds) over the course of a single I-V sweep.

4.7 Data Aggregation

The results of each solve were collected into MATLAB `table` objects (e.g. `DATA_SET`, `DATA_RESET`, `DATA_OFF`, `I_V`) and progressively concatenated using `vertcat` across the minimization iterations, voltage/current sweep steps, and switching cycles, before being saved to disk for post-processing:

```
save('Complete_I_V_avg_DATA.mat', 'I_V', 'DATA_OFF', 'DATA_min_SET', ...  
    'DATA_SET', 'DATA_ON', 'DATA_ON_neg', 'DATA_min_RESET', ...  
    'DATA_RESET', 'DATA_OFF_neg');
```

This saved MAT-file forms the input to the post-processing and I-V plotting stage described in Chapter 7.

4.8 Summary

The MATLAB–COMSOL LiveLink workflow described in this chapter transformed the four static COMSOL device modules into a fully automated, parameter-driven simulation pipeline capable of tracing out a complete RRAM switching cycle — SET, ON, RESET, and OFF — through repeated, programmatically controlled finite-element solves. Establishing and debugging this workflow reliably, however, required resolving a substantial number of integration issues between the inherited COMSOL model exports and the MATLAB control scripts, which are documented in detail in Chapter 5.

Chapter 5

Debugging the Inherited Simulation Pipeline

5.1 Overview

A substantial part of this internship was devoted to debugging the inherited RRAM MATLAB–COMSOL codebase before it could be run successfully. The original `.m` files had been generated from a COMSOL model whose geometry had been actively edited over time, and the accompanying MATLAB code had, at some point, passed through a text/document conversion process that introduced additional errors unrelated to COMSOL itself. This chapter documents the debugging process in the order the issues were encountered, as the process itself forms a significant part of the practical learning outcome of this internship.

5.2 MATLAB Syntax Errors from Word-Wrapped Comments

The first class of errors encountered were plain MATLAB syntax errors of the form `Unrecognized function or variable 'X'`, where `X` was not a variable used anywhere else in the script. On inspection, each instance traced back to a multi-line comment that had been word-wrapped across two lines, where only the first line retained its leading `%` character:

```
%sets the Source voltage and filament radius in COMSOL file and  
solves the problem
```

Here, `solves the problem` — missing its `%` — was interpreted by MATLAB as an executable statement rather than a comment, causing it to fail with `Unrecognized function or variable 'solves'`. This exact pattern recurred at several points throughout the driver scripts (`Complete_I_V_C2C.m`, `Complete_I_V_avg.m`) and was resolved in each case by merging the wrapped text back into a single commented line.

5.3 Corrupted Geometry Parameter Expression

A second, related but distinct error occurred inside the `RRAM_OFF` module during geometry construction: Unknown model parameter. Parameter: `h_dih_g`

Investigation traced this to a geometry size expression that should have read `'h_di-h_g'` (dielectric height minus gap height) but had lost its minus sign during the same text-conversion process, becoming the nonexistent parameter name `h_dih_g`. This was corrected by restoring the missing hyphen in the affected `.set('size', ...)` call.

5.4 Mesh Selection Domain Mismatch

A more structural issue arose when running `RRAM_SET`, which failed with a `COMSOL SelectionOutOfBoundsException: Illegal input vector illegal entity number`. This error originated from hardcoded domain-number arrays in the mesh-generation section (e.g. `selection.set([1 6 8])`) that no longer matched the actual number and ordering of domains in the current geometry. Comparing the current COMSOL geometry tree (obtained via screenshot) against the MATLAB-generated code revealed that the model's geometry sequence had been edited — rectangles added, removed, and relabeled — since the code had last been generated, changing the domain count from eight (`r1–r8`) to a non-contiguous set of tags (`r2, r5, r7, r8, r9, r10, r11, r12`).

Rather than attempting to hand-patch the stale hardcoded domain indices, all four modules (`RRAM_OFF`, `RRAM_ON`, `RRAM_SET`, `RRAM_RESET`) were **re-exported directly from the live COMSOL Desktop model** (File → Save As → Model File for MATLAB), ensuring that the generated MATLAB code reflected the true, current geometry and domain numbering.

5.5 Verifying Domain Numbering Across Modules

Following the re-export, the domain numbers referenced throughout the MATLAB post-processing code (filament, dielectric, Hf layer, electrodes, and — for the OFF/RESET modules — the switching gap) were systematically verified against COMSOL's Selection List and Materials overview panel for each of the four newly exported modules. This confirmed that:

- The SET and ON modules (which contain no gap domain) shared identical domain numbering: filament = 3, Hf layer = 4, electrodes = [2, 5], dielectric = 7, SiO₂ = [1, 6, 8].
- The OFF and RESET modules (which include an additional "Gap" domain inserted during geometry construction) had every domain number from 3 onward shifted by one relative to SET/ON: gap = 3, filament = 4, Hf layer = 5, electrodes = [2, 6], dielectric = 8, SiO₂ = [1, 7, 9].

Cross-checking this against the existing hardcoded selection numbers in the driver scripts (`Complete_I_V_avg.m`, `Complete_I_V_ramprate.m`, `Complete_I_V_C2C.m`) confirmed that these scripts had, in fact, already been written correctly for this shifted numbering scheme — meaning no changes were required to the driver scripts themselves, only to the four re-exported model files.

5.6 Invalid Blank Mesh Parameter

Running the newly re-exported `RRAM_SET` module produced a further COMSOL error: `Invalid property value – Property: hmax ... 'Maximum element size' is a real number... x > 0`. This was traced to a leftover command from an interactive mesh-sizing trial-and-error session in COMSOL Desktop:

```
model.component('comp1').mesh('mesh1').feature('ftri1').feature('size1').set('hmax', "");
```

Setting `hmax` to an empty string is harmless when performed interactively mid-edit in the GUI, but is invalid when replayed as a scripted LiveLink command, since the property is committed immediately. As the very next line in the script disabled the custom element-size setting entirely (`hmaxactive = false`), this line was determined to be safely removable without affecting the mesh configuration.

5.7 Duplicate COMSOL Parameter Name

A further error, `Duplicate parameter/variable name – Variable: R`, occurred during model compilation when the MATLAB driver script set a global COMSOL parameter named `R` (used to represent the voltage ramp-rate) via `model_OFF.param.set('R', ...)`. This name collided with an existing symbol already defined within the model. Since `R` is a common, easily-colliding identifier, the parameter name was changed to a more distinctive name (`ramp_R`) consistently across all four model instances (`model_OFF`, `model_ON`, `model_ON_neg`, `model_OFF_neg`) that set this parameter.

5.8 Inconsistent Table Row Counts from a Residual Parametric Sweep

Running the corrected pipeline further surfaced a MATLAB error, `All table variables must have the same number of rows`, when constructing result tables inside the SET free-energy function. This was traced to a residual **Auxiliary Sweep** configuration left active on the SET module's Stationary study (`useparam = true`, sweeping the source current `I_S` over 14 values internally), inherited from an

earlier interactive parameter-sweep exploration in COMSOL Desktop. Because the MATLAB driver script already performed its own current sweep externally, this internal COMSOL sweep caused every post-processing extraction (`mphglobal`, `mphmean`, `mphmax`) to return a 14-element vector instead of the expected scalar, conflicting with the single-valued quantities being combined into the same table. Disabling the internal sweep (`useparam = false`) resolved the mismatch.

5.9 COMSOL Server Memory Exhaustion

The final, and most significant, issue encountered was an intermittent `Not connected to a server` error partway through longer runs of the driver scripts. This was identified as a **server-side memory exhaustion** issue: each call to `RRAM_OFF`, `RRAM_SET`, `RRAM_ON`, or `RRAM_RESET` creates an entirely new COMSOL model object on the background server via `ModelUtil.create`, and since the driver scripts call these functions repeatedly inside nested sweep and cycle loops without ever releasing the resulting model objects, the accumulated memory footprint eventually exceeded the server's capacity and the connection was dropped.

The fix identified was to explicitly release each model object from server memory, using `ModelUtil.remove`, immediately after all required quantities had been extracted from it and before the next model instance was built:

```
model_OFF = RRAM_OFF;  
% ... extract required quantities via mphglobal / mphmean / mphmax ...  
ModelUtil.remove(model_OFF.tag);
```

This cleanup was identified as necessary at the end of each model's usage within every cycle iteration of the driver scripts.

5.10 Summary

Table 5.1 summarizes the debugging issues encountered and resolved during this internship.

Table 5.1: Summary of debugging issues resolved

Issue	Root Cause	Resolution
Unrecognized function or variable	Wrapped comment missing %	Merged/re-commented affected lines
Unknown model parameter: h_dih_g	Corrupted geometry expression	Restored missing hyphen
SelectionOutOfBoundsExcep tion	Stale hardcoded domain numbers	Re-exported all four modules from live COMSOL model
Domain number verification	Gap domain shifts numbering in OFF/RESET	Verified via Selection List / Materials overview
Invalid property value: hmax	Blank value from interactive mesh trial	Removed redundant <code>set('hmax','')</code> line
Duplicate parameter/variable name: R	Naming collision	Renamed parameter to <code>ramp_R</code>
All table variables must have the same number of rows	Residual internal parametric sweep	Disabled <code>useparam</code> on SET study
Not connected to a server	Server memory exhaustion	Added <code>ModelUtil.remove</code> after each model's use

This structured debugging process — reading COMSOL and MATLAB error traces, isolating root causes, and verifying fixes against the live COMSOL model — constituted a significant and instructive component of the internship, and directly enabled the simulation results presented in Chapter 6.

Chapter 6

Results

6.1 Current Status of the Simulation Pipeline

Following the debugging process described in Chapter 5, all four COMSOL modules — `RRAM_OFF`, `RRAM_ON`, `RRAM_SET`, and `RRAM_RESET` — now build and solve successfully within the MATLAB–COMSOL LiveLink environment. The driver script `Complete_I_V_avg.m` executes the full switching cycle structure — OFF, SET, ON, ON (negative polarity), RESET, and OFF (negative polarity) — in sequence, calling each COMSOL module in turn, performing the free-energy minimization sweep described in Section 2.2 for the SET and RESET stages, and aggregating the resulting data into MATLAB tables.

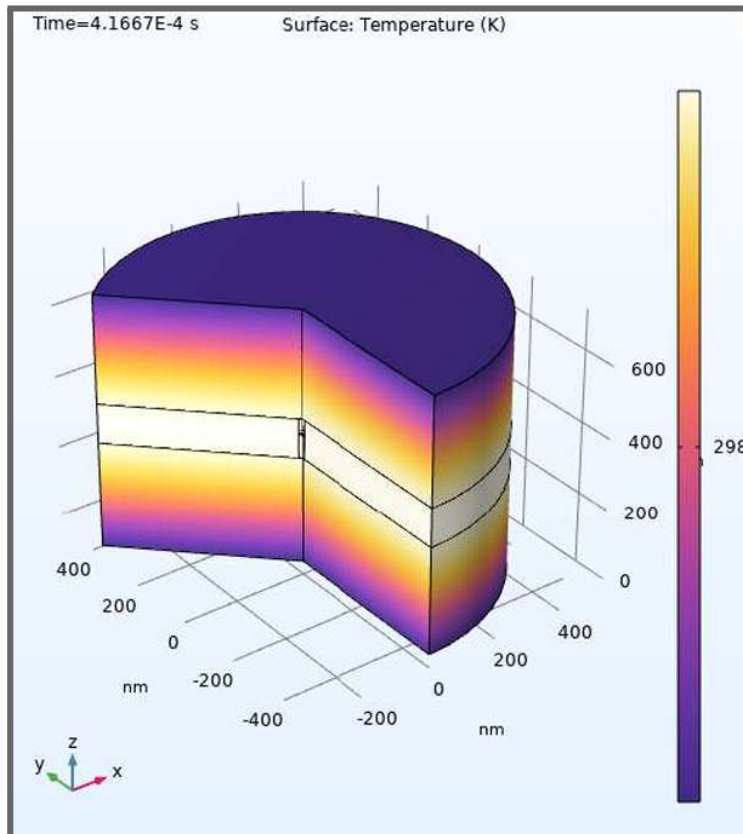


Figure 6.1: Example 3D temperature distribution output from the `RRAM_OFF` module (revolved 2D axisymmetric solution), illustrating the model's post-processing visualization capability.

6.2 Data Generated

Each run of `Complete_I_V_avg.m` produces the following tables, saved to `Complete_I_V_avg_DATA.mat`:

Table 6.1: Output tables generated per simulation run

Table	Contents
<code>DATA_OFF / DATA_OFF_neg</code>	Time-resolved device/source/load voltage, current, and gap temperature during the OFF-state current sweep
<code>DATA_SET</code>	Every evaluated point during the SET free-energy sweep — filament radius, current, voltage, resistance, filament/dielectric temperature, and all individual free-energy terms
<code>DATA_min_SET</code>	The converged (minimum free-energy) SET points only, used for the final I-V trace
<code>DATA_ON / DATA_ON_neg</code>	Time-resolved device characteristics during the ON-state voltage ramp, positive and negative polarity
<code>DATA_RESET</code>	Every evaluated point during the RESET free-energy sweep — gap length, filament/gap/dielectric temperature, and free-energy terms
<code>DATA_min_RESET</code>	The converged RESET points only
<code>I_V</code>	The combined device voltage–current pairs assembled across all six stages, forming the final I-V characteristic

This table structure mirrors the physical sequence of a single switching cycle and allows each stage to be inspected independently, as well as combined into the overall I-V curve.

6.3 Post-Processing Infrastructure

A dedicated post-processing script, `plot_RRAM_IV.m`, was developed to load a saved `.mat` file and generate three complementary views of the data:

1. A **combined I-V scatter plot**, overlaying every stage and cycle in a single view.
2. A **stage-wise plot**, showing OFF, SET, ON, ON(neg), RESET, and OFF(neg) as separate labelled traces on one hysteresis-style axis.
3. A **log-scale stage-wise plot** of the same data, plotting the absolute current on a logarithmic axis — a format commonly used in RRAM literature (e.g. Fantini et al., IMEC IWM 2012) to more clearly show the multi-decade current change across the switching transition.

The script additionally includes a fallback for the RESET stage: if `DATA_min_RESET` is empty (indicating the minimization did not converge), it automatically falls back to plotting the raw, unconverged `DATA_RESET` sweep points instead, clearly labelled as such, so that the attempted RESET trajectory remains visible for diagnostic purposes even when a stable solution was not found.

6.4 Preliminary Observations

Running this pipeline end-to-end surfaced two outstanding physical/numerical issues that must be resolved before the I-V characteristics can be considered validated against the reference literature:

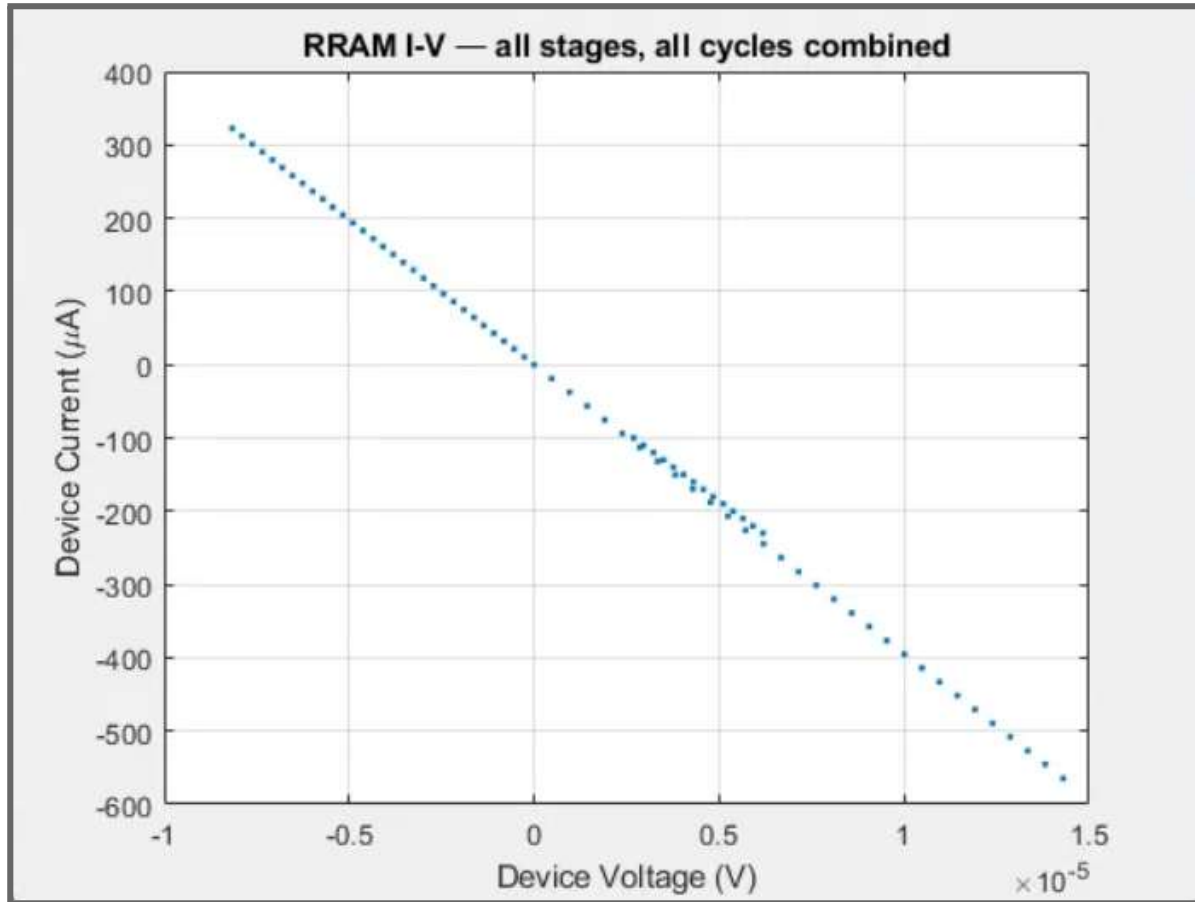


Figure 6.2: Combined device I-V scatter plot generated by `plot_RRAM_IV.m`, showing all stages and cycles from a single run of `Complete_I_V_avg.m`. Note the device voltage axis spans only $\sim \pm 1.5 \times 10^{-5}$ V — several orders of magnitude smaller than the applied source voltage amplitudes (1.25 V / -1.75 V), consistent with the device voltage scaling issue described below.

(a) Non-convergent RESET minimization: In the run examined, `DATA_min_RESET` was returned empty, meaning the free-energy minimization over the trial gap-length range did not locate a stable minimum for the tested RESET voltage steps. This could result from the initial gap-length search range being too narrow relative to where the true minimum lies, an issue in one of the RESET free-energy terms causing the free-energy curve to be monotonic (and therefore have no interior minimum) over the searched range, or a mismatch between the RESET starting filament radius (carried over from the preceding SET stage) and the assumptions of the RESET energy model.

(b) Anomalous device voltage scale: The device voltage values extracted via `mphglobal(model, 'cir.IvsU1_v')` were found to be several orders of magnitude smaller than the applied source voltage amplitudes defined in the script (≈ 1.25 V for SET/ON, -1.75 V for RESET). Because this value is read directly from the COMSOL circuit solution rather than computed or rescaled anywhere in the MATLAB post-processing code, this points to an issue upstream in the model itself — most likely in how the `IvsU1` Model-Device I-V circuit element is wired to the `ec` physics terminal, in the load resistor value/units, or in the terminal boundary condition definition — rather than a plotting or unit-conversion error in the MATLAB script.

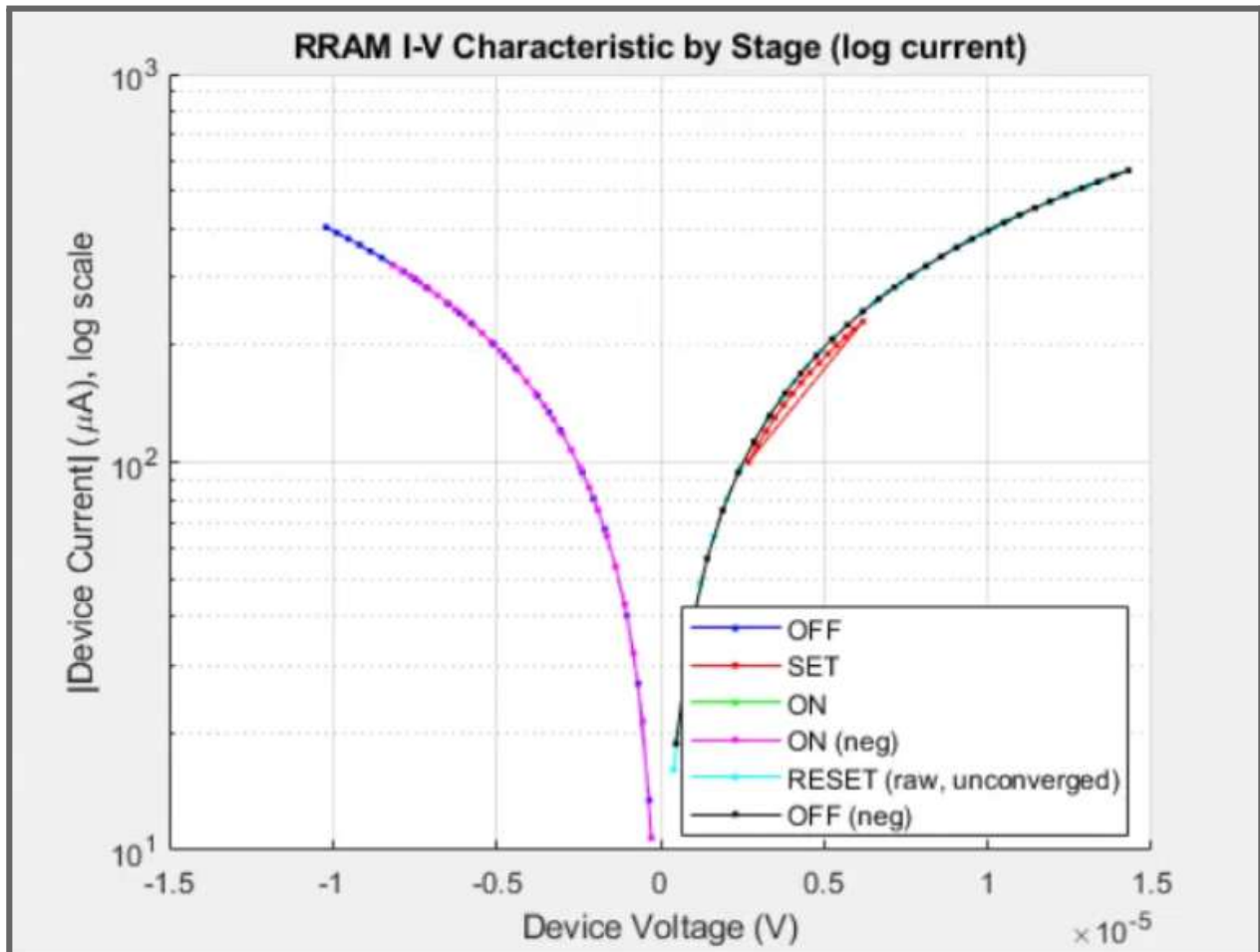


Figure 6.3: Stage-wise I-V characteristic (log current scale) generated by `plot_RRAM_IV.m`. The RESET stage is explicitly labelled "RESET (raw, unconverged)" in the legend, reflecting the fallback behaviour described in Section 7.4, since `DATA_min_RESET` was returned empty for this run. The device voltage axis again shows the anomalous $\sim 10^{-5}$ V scale discussed in Section 8.3.

6.5 Interpretation

Both observations above are consistent with the simulation pipeline being **structurally functional but not yet physically validated**: every COMSOL module builds, solves, and returns results without runtime errors, and the full data-collection and plotting infrastructure is in place and operating correctly on whatever data is produced. What remains is to trace these two issues to their root cause within the circuit and free-energy model configuration, which is the immediate next step outlined in Chapter 9.

Chapter 7

Data Processing and I-V Characterization

7.1 Purpose of a Separate Post-Processing Stage

The simulation pipeline described in Chapters 4–6 was deliberately structured so that **simulation** and **visualization** are handled by separate scripts. `Complete_I_V_avg.m` (and its counterparts, `Complete_I_V_ramprate.m` and `Complete_I_V_C2C.m`) are solely responsible for driving COMSOL and saving raw results to disk; a separate function, `plot_RRAM_IV.m`, is solely responsible for reading a saved `.mat` file and producing plots. This separation was a deliberate design decision made during the internship, for two practical reasons: it allows a completed simulation run to be re-plotted, re-formatted, or re-examined without re-running the (computationally expensive) COMSOL solves, and it makes debugging easier, since a plotting issue can immediately be ruled in or out as a plotting-script problem versus a simulation-data problem simply by inspecting the saved table values directly.

7.2 Structure of `plot_RRAM_IV.m`

The script accepts the filename of a saved results `.mat` file as input, defaulting to `Complete_I_V_avg_DATA.mat` if none is given:

```
function plot_RRAM_IV(matfile)
if nargin < 1
    matfile = 'Complete_I_V_avg_DATA.mat';
end
S = load(matfile);
```

Before plotting anything, the script prints a summary of every table found in the file — its name, dimensions, and column names — to the MATLAB console. This diagnostic step was included specifically to guard against a recurring source of error throughout this project: table column names in the driver scripts were not always consistent or obviously named (e.g. `device_Voltage_V1` for SET vs. `device_Voltage_V2` for RESET), so printing them explicitly before use avoids silent mismatches.

7.3 Combining Stage-Wise Data into a Full I-V Curve

Each stage of a switching cycle — OFF, SET, ON, ON(neg), RESET, OFF(neg) — is stored in its own table with its own column-naming convention, reflecting how each stage was computed differently within the driver script (time-resolved for OFF/ON, per-minimization-step for SET/RESET). A helper function, `plotStage`, was written to handle this uniformly: given a table name and its voltage/current column names, it checks that both the table and the expected columns exist, and if so, adds that stage's trace to the current plot with an appropriate legend label. This was implemented defensively (with `warning` rather than `error` on a missing table/column) so that a single missing or malformed table — for instance, an empty `DATA_min_RESET`, as discussed in Section 6.4 — does not prevent the remaining stages from being plotted.

7.4 Handling Non-Convergent RESET Data

Because the RESET free-energy minimization did not converge in the run examined (Section 6.4a), a dedicated fallback function, `plotResetWithFallback`, was written specifically for the RESET stage. It first attempts to plot `DATA_min_RESET` (the converged, minimized points); if this table is empty, it instead plots the raw `DATA_RESET` table — which contains every trial gap-length point evaluated during the (unconverged) sweep — and relabels the trace as **"RESET (raw, unconverged)"** in the legend, with an accompanying console warning. This ensures that the attempted RESET trajectory remains visible on the plot for diagnostic purposes, rather than silently disappearing, while making unmistakably clear to anyone reading the plot that this particular trace does not represent a converged, physically validated result.

7.5 Three Plotting Modes

The script produces three separate figures from the same underlying data:

1. **Combined I-V scatter** — every stage and every cycle, plotted together as a single scatter, giving an unfiltered view of the complete dataset.
2. **Stage-wise linear plot** — the same data, but split into six clearly labelled, colour-coded traces (one per switching stage), plotted on a linear current axis.
3. **Stage-wise log-scale plot** — identical to (2), but with the absolute current plotted on a logarithmic axis, which is the format typically used in RRAM literature (e.g. the Fantini et al. reference figures shown in Section 3.IX of the internship reference material) to make the multiple-decade current change across the resistive switching transition visible on a single axis.

7.6 Present Limitation

At the time of writing, this post-processing infrastructure has been implemented and verified to run correctly against the structure of the saved data tables; however, since the underlying simulation data itself has not yet been validated (Section 6.4), the resulting plots have not been included in this report. Generating and validating these plots against the reference ramp-rate-dependence and cycle-to-cycle-variation characteristics (Section 3.IX of the reference material) is the immediate next step of this work, described further in Chapter 9.

Chapter 8

Current Status and Limitations

8.1 Summary of Current Status

At the time of writing, the RRAM modelling pipeline developed during this internship is in the following state:

- All four COMSOL modules (`RRAM_OFF`, `RRAM_ON`, `RRAM_SET`, `RRAM_RESET`) build and solve without errors, following the geometry and parameter-naming fixes described in Chapter 5.
- The MATLAB–COMSOL LiveLink workflow runs end-to-end through a complete switching cycle without crashing, following the server memory-management fix described in Section 5.9.
- The data-aggregation and post-processing infrastructure (`Complete_I_V_avg.m` and `plot_RRAM_IV.m`) is functional and correctly structured to produce I-V characteristics once valid simulation data is available.

What remains incomplete is the **physical validation** of the output — the I-V characteristics have not yet been confirmed to reproduce the expected RRAM switching behaviour reported in the reference literature.

8.2 Limitation 1: Non-Convergent RESET Minimization

As discussed in Section 6.4, the free-energy minimization for the RESET stage did not converge to a stable gap length in the run examined. This limitation means that the RESET branch of the I-V curve — and consequently the full switching cycle, since each cycle's starting condition depends on the previous RESET outcome — cannot currently be trusted as physically representative. Locating the root cause will require inspecting the individual free-energy terms (`Thermal_energy_*`, `Electro_Static_Energy_eV`, `Surface_Energy_eV`, `Volume_Energy_eV`) computed within `FE_RESET` across the trial gap-length range to determine whether the free-energy curve genuinely lacks an interior minimum under the tested conditions, or whether the search range/initial bracketing was inadequate.

8.3 Limitation 2: Device Voltage Scale

As discussed in Section 6.4, the device voltage extracted from the COMSOL circuit solution does not match the expected order of magnitude of the applied source voltage. Since this quantity is read directly from COMSOL's `cir.IvsU1_v` circuit variable rather than computed in MATLAB, this limitation points to the model's electrical circuit configuration itself — most plausibly the `IvsU1` Model-Device I-V element's connection to the `ec` terminal, the load resistor's value or units, or the terminal boundary condition — rather than to any of the MATLAB scripts described in this report.

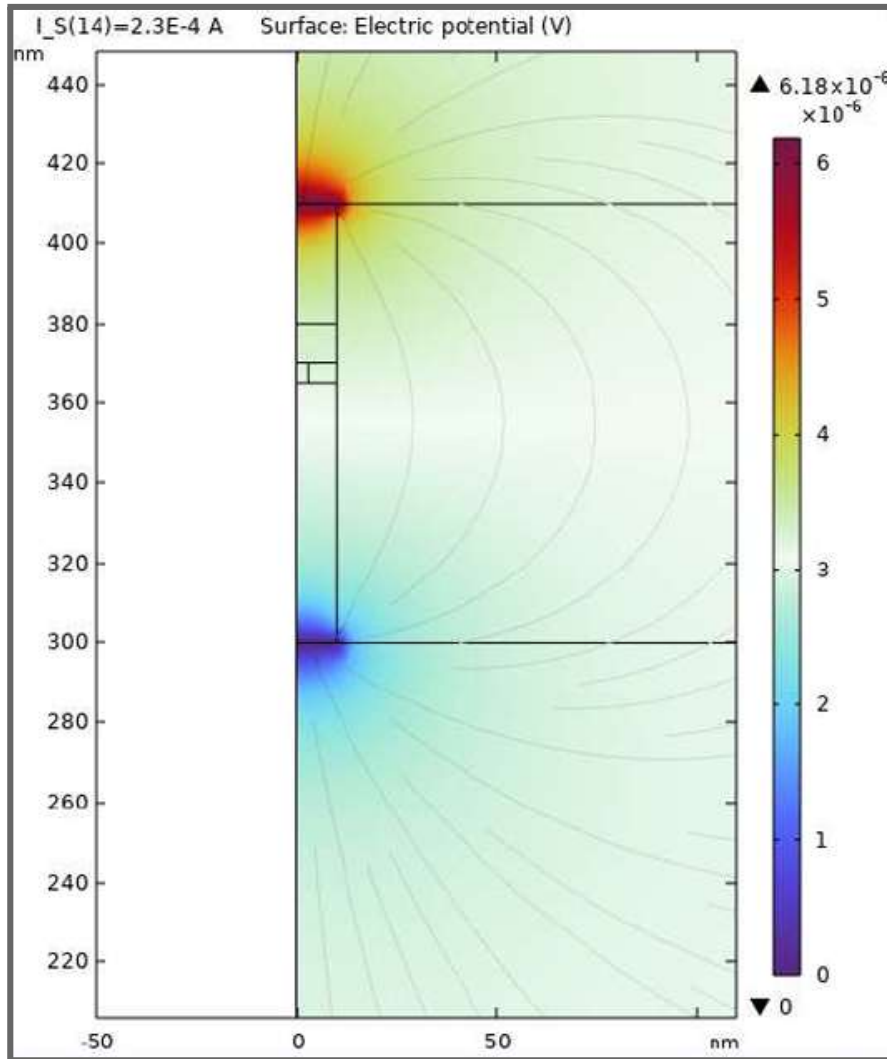


Figure 8.1: Electric potential distribution from the `RRAM_SET` module, showing a potential range on the order of 10^{-6} V despite an applied source current corresponding to a much larger expected terminal voltage — consistent with the device-voltage scaling discrepancy discussed in Section 8.3.

8.4 Limitation 3: Time Constraints on Full Validation

Given the internship duration, the debugging effort documented in Chapter 5 consumed the majority of the available time, leaving limited time to complete the ramp-rate-dependence and cycle-to-cycle-variation studies (`Complete_I_V_ramprate.m` and `Complete_I_V_C2C.m`) that were intended as the final validation benchmarks against the reference results (Section 3.IX). These scripts share the same underlying model and are expected to run once the two limitations above are resolved, but this has not yet been confirmed.

8.5 Scope of This Report

Given the above, this report is presented as documentation of a **working, debugged simulation infrastructure**, together with a clear, specific list of the remaining issues standing between the current state and a fully validated result — rather than as a report of completed and validated RRAM I-V characteristics. This distinction is made explicit throughout the report, consistent with the Abstract and Declaration.

Chapter 9

Learning Outcomes and Future Scope

9.1 Technical Learning Outcomes

This internship provided practical, hands-on experience across several areas that extend beyond simply operating a simulation tool:

Multiphysics finite-element modelling: Working with the RRAM device model required understanding how a physical device geometry is translated into a finite-element representation, how distinct physics interfaces (Heat Transfer in Solids, Electric Currents) are coupled through a multiphysics node (Electromagnetic Heating), and how an external lumped-element circuit is integrated with a distributed finite-element domain through a terminal boundary condition.

Programmatic control of a simulation environment: Learning to drive COMSOL entirely through MATLAB via LiveLink — rather than through the Desktop GUI — required understanding COMSOL's underlying object model (components, physics, studies, solvers, results) well enough to manipulate it programmatically, and reinforced the importance of writing simulation scripts that separate model-building, solving, and post-processing into distinct, inspectable stages.

Systematic debugging of a multi-layered pipeline: The debugging process documented in Chapter 5 required distinguishing between several categories of failure that could all present as a MATLAB error at the command line but had entirely different root causes: plain MATLAB syntax errors, corrupted COMSOL model expressions, structural mismatches between a live model and previously exported code, invalid scripted values that were only harmless when entered interactively, naming collisions inside the COMSOL model itself, residual configuration left over from unrelated interactive exploration, and server-side resource management. Learning to read a COMSOL Java exception trace, isolate it to a specific `.set(...)` call, and verify the fix against the live model tree (rather than by assumption) was a central and repeated skill developed throughout this internship.

The importance of intermediate checkpoints: Perhaps the most important lesson of this internship was that a simulation pipeline running "without errors" and a simulation pipeline producing "correct results" are two different claims. The final I-V plot alone would not have revealed *why* the RESET stage failed to converge or *why* the device voltage was orders of magnitude too small — these were only identified by inspecting

the intermediate MATLAB tables (`DATA_RESET`, `I_V`) directly. This reinforced the value of structuring a simulation pipeline so that every intermediate quantity remains inspectable, rather than only exposing a final aggregated result.

9.2 Future Scope

The following steps are identified as the immediate continuation of this work:

1. **Resolve the RESET convergence issue** (Section 8.2) by examining the individual free-energy terms computed by `FE_RESET` across the trial gap-length range, and adjusting the search range or investigating the term(s) responsible for the absence of a well-defined minimum.
2. **Resolve the device voltage scaling issue** (Section 8.3) by inspecting the Electrical Circuit (`cir`) physics configuration in COMSOL Desktop — specifically the `IvsU1` Model-Device I-V element's connection to the `ec` terminal, the units and value of the load resistor `R1`, and the terminal boundary condition definition.
3. **Generate and validate the baseline I-V characteristic** using `Complete_I_V_avg.m` once the above two issues are resolved, and confirm that it reproduces the qualitative bipolar switching hysteresis expected of the device.
4. **Run the voltage ramp-rate dependence study** (`Complete_I_V_ramprate.m`) across multiple ramp rates (e.g. 100 V/s, 10 kV/s, 1 MV/s) and compare the resulting family of I-V curves against the corresponding reference figure (Section 3.IX, "Ramp-rate dependence").
5. **Run the cycle-to-cycle variation study** (`Complete_I_V_C2C.m`) across a sufficient number of switching cycles and compare the resulting spread of I-V curves against the reference figure (Section 3.IX, "Cycle-to-Cycle Variation").
6. **Quantitatively benchmark** the converged model against the reference SET/RESET voltages, ON/OFF resistance ratio, and switching threshold trends reported by Niraula and Karpov, once qualitative agreement is achieved.

Chapter 10

Conclusion

This internship set out to develop and validate a numerical model of filamentary RRAM switching using COMSOL Multiphysics 6.0 coupled with MATLAB via LiveLink, following the thermodynamic free-energy minimization framework proposed by Niraula and Karpov. The primary deliverable achieved was a fully debugged, end-to-end simulation pipeline: all four COMSOL device-state modules (OFF, ON, SET, RESET) now build and solve without error, and the MATLAB driver script executes a complete switching cycle from start to finish, supported by a working post-processing framework for generating I-V characteristics. This was not the state of the codebase at the start of the internship — reaching it required identifying and resolving eight distinct classes of errors spanning MATLAB syntax issues, corrupted COMSOL parameters, stale geometry selections, an invalid mesh property, a parameter naming collision, a residual internal sweep, and a COMSOL server memory leak, as detailed in Chapter 5.

For the research group, this means the RRAM modelling framework — previously blocked at the point of simply running without crashing — is now in a state where the remaining work is narrow and well-defined rather than open-ended. The two outstanding issues, a non-convergent RESET minimization and a device voltage extracted from the circuit solution that does not match the applied source amplitude, have each been traced to a specific, identifiable part of the model rather than left as unexplained discrepancies, which should substantially shorten the time needed to reach validated results. On a personal level, this internship gave me hands-on grounding in multiphysics finite-element modelling, in controlling a simulation environment programmatically rather than through a GUI, and in reading and reasoning through unfamiliar error traces methodically rather than by guesswork — a skill I expect to carry well beyond this specific project.

With the internship period concluded, the immediate next steps are to resolve the two outstanding issues described in Chapter 8 by inspecting the RESET free-energy terms and the circuit's terminal voltage configuration respectively, generate a validated baseline I-V characteristic using the now-working pipeline, and then run the voltage ramp-rate and cycle-to-cycle variation studies to benchmark the model against the reference results that originally motivated this work.

REFERENCES

- [1] D. Niraula and V. G. Karpov, "Comprehensive numerical modeling of filamentary RRAM devices including voltage ramp-rate and cycle-to-cycle variations," *Journal of Applied Physics*, vol. 124, no. 7, 2018. arXiv:1806.01397v2.
- [2] Dipesh Niraula and Victor Karpov, *RRAM Device Numerical Model — COMSOL/MATLAB Implementation Guide*, University of Toledo, 2018.
- [3] A. Fantini et al., "Intrinsic switching variability in HfO₂ RRAM," in *2013 5th IEEE International Memory Workshop (IMW)*, IEEE, 2013.
- [4] H.-S. P. Wong, H.-Y. Lee, S. Yu, Y.-S. Chen, Y. Wu, P.-S. Chen, B. Lee, F. T. Chen, and M.-J. Tsai, "Metal-oxide RRAM," *Proceedings of the IEEE*, vol. 100, no. 6, pp. 1951–1970, 2012.
- [5] R. Waser and M. Aono, "Nanoionics-based resistive switching memories," *Nature Materials*, vol. 6, pp. 833–840, 2007.
- [6] C. Walczyk et al., "Impact of temperature on the resistive switching behavior of embedded HfO₂-based RRAM devices," *IEEE Transactions on Electron Devices*, vol. 58, no. 9, pp. 3124–3131, 2011.
- [7] M. Lanza et al., "Grain boundaries as preferential sites for resistive switching in the HfO₂ resistive random access memory structures," *Applied Physics Letters*, vol. 100, p. 123508, 2012.
- [8] D. Duncan, B. Magyari-Köpe, and Y. Nishi, "Filament-induced anisotropic oxygen vacancy diffusion and charge trapping effects in hafnium oxide RRAM," *IEEE Electron Device Letters*, vol. 37, no. 4, pp. 400–403, 2016.
- [9] S. Dirkmann, J. Kaiser, C. Wenger, and T. Mussenbrock, "Filament growth and resistive switching in hafnium oxide memristive devices," *ACS Applied Materials & Interfaces*, vol. 10, no. 17, pp. 14857–14868, 2018.

- [10] X. Ding, Y. Feng, P. Huang, L. Liu, and J. Kang, "Low-power resistive switching characteristic in HfO_2/TiOx bi-layer resistive random-access memory," *Nanoscale Research Letters*, vol. 14, no. 1, p. 157, 2019.
- [11] Y. Y. Chen et al., "Balancing SET/RESET pulse for $>10^{10}$ endurance in HfO_2/Hf 1T1R bipolar RRAM," *IEEE Transactions on Electron Devices*, vol. 59, no. 12, pp. 3243–3249, 2012.
- [12] COMSOL Multiphysics® v. 6.0, COMSOL AB, Stockholm, Sweden. www.comsol.com.
- [13] *Introduction to LiveLink™ for MATLAB®*, COMSOL AB, Stockholm, Sweden, 2023.
- [14] COMSOL, "Run COMSOL Multiphysics® simulations with MATLAB®," COMSOL AB. [Online]. Available: <https://www.comsol.com/livelink-for-matlab>
- [15] O. Ondrejka et al., "Case studies on optimization problems in MATLAB and COMSOL Multiphysics by means of the LiveLink," in *AIP Conference Proceedings*, vol. 1738, 2016.
- [16] Zahoor, F., Azni Zulkifli, T.Z. & Khanday, F.A. Resistive Random Access Memory (RRAM): an Overview of Materials, Switching Mechanism, Performance, Multilevel Cell (mlc) Storage, Modeling, and Applications. *Nanoscale Res Lett* **15**, 90 (2020).
- [17] Zahoor, F., Hussin, F.A., Isyaku, U.B. *et al.* Resistive random access memory: introduction to device mechanism, materials and application to neuromorphic computing. *Discover Nano* **18**, 36 (2023).